

# C++ COMPETITIVE PROGRAMMING + DATA STRUCTURES + ALGORITHMS MASTER TEXTBOOK

## PART X

# Recursion and Backtracking

*The call stack made visible — base cases, recursive cases, recursion trees, and how backtracking systematically explores every possibility without missing or repeating one*

**PART X**

# Recursion and Backtracking

*A function that solves a problem by calling itself on a smaller version of the same problem*

Part IX opened with the call stack as the reason function calls behave the way they do — the most recently called function is the first one to finish. This Part studies that stack directly, through the lens of recursion: functions that call themselves. Once recursion is solid, backtracking follows almost for free — it's recursion with one extra discipline: try a choice, recurse, then undo the choice and try the next one. That one discipline is how you generate every permutation, every subset, and every valid N-Queens board without writing a single nested loop by hand.

## Contents of Part X

- 1. Recursion: Base Case, Recursive Case, and the Call Stack
- 2. Classic Recursion Patterns
- 3. Recursion on Arrays and Strings
- 4. Complexity of Recursion
- 5. Backtracking Fundamentals — Subsets and Subsequences
- 6. Permutations and Combinations
- 7. N-Queens and the Art of Pruning
- Part X Revision, Cheat Sheet, and Interview Questions

# Chapter 1 — Recursion: Base Case, Recursive Case, and the Call Stack

## 1.1 What recursion actually is

In one sentence: recursion is solving a problem by expressing it in terms of a smaller instance of the exact same problem, plus a starting point simple enough to answer directly.

Every correct recursive function needs exactly two parts. The base case is the smallest input the function can answer without recursing further — it stops the recursion. The recursive case is where the function calls itself on a smaller or simpler input, and combines that result to answer the current input. Miss the base case, or fail to shrink the input toward it, and the function calls itself forever (in practice: until it crashes).

```
int factorial(int n) {  
    if (n <= 1) return 1;           // base case  
    return n * factorial(n - 1);    // recursive case: smaller  
    input (n-1)  
}
```

## 1.2 The call stack, made visible

Every function call — recursive or not — pushes a new stack frame onto the call stack (Part IX, Chapter 1): a chunk of memory holding that call's local variables, parameters, and the address to return to. A recursive call is no different from any other call in this respect — the only unusual thing is that the function calling it and the function being called happen to share the same name.

Tracing `factorial(4)` by hand:

| Call                      | Waiting on                    | Stack depth |
|---------------------------|-------------------------------|-------------|
| <code>factorial(4)</code> | <code>4 * factorial(3)</code> | 1           |
| <code>factorial(3)</code> | <code>3 * factorial(2)</code> | 2           |

| Call         | Waiting on                             | Stack depth |
|--------------|----------------------------------------|-------------|
| factorial(2) | 2 * factorial(1)                       | 3           |
| factorial(1) | returns 1 (base case, no further call) | 4           |

Once `factorial(1)` returns 1, the calls resolve in reverse order — last called, first to finish, exactly the LIFO behaviour from Part IX: `factorial(2)` returns  $2*1=2$ , `factorial(3)` returns  $3*2=6$ , `factorial(4)` returns  $4*6=24$ . Nothing “goes back in time” — every waiting call simply stays parked on the stack, holding its own copy of  $n$ , until the call above it returns a value it can use.

⚠ **Stack overflow is a real, common bug**

The call stack has a finite size (typically a few megabytes by default). A recursive function with a missing or unreachable base case — or one that simply recurses too deep for the input size — exhausts that space and crashes with a stack overflow. This is a genuine, frequent cause of runtime errors on judges when recursion depth scales with  $n$  and  $n$  is large (e.g. recursing  $10^6$  levels deep on a plain linked list or a skewed tree).

## 1.3 Direct vs. indirect recursion

Most recursion you'll write is direct: a function calls itself by name, as above. Indirect recursion is when function A calls function B, which calls function A again — the cycle is still recursion, just spread across two or more functions. The same base-case/recursive-case discipline applies either way; indirect recursion is rare in competitive programming and mentioned here mainly so the term isn't unfamiliar if it comes up.

 **Quick Revision**

- Every recursive function needs a base case (stops it) and a recursive case (shrinks the input and calls itself).
- Each call gets its own stack frame with its own copy of local variables and parameters — calls don't share state unless it's passed by reference or is global.
- Calls resolve in reverse order of when they were made — the deepest call finishes first, exactly like a stack's LIFO order.
- Recursion depth is bounded by stack size; very deep recursion (proportional to a large  $n$ ) risks a stack overflow.

## Chapter 2 — Classic Recursion Patterns

### 2.1 Fibonacci — and why naive recursion can be slow

The Fibonacci sequence (each term is the sum of the two before it) is the standard example for showing that recursion, written carelessly, can be extremely wasteful.

```
long long fib(int n) {  
    if (n <= 1) return n;           // base cases: fib(0)=0,  
    fib(1)=1  
    return fib(n - 1) + fib(n - 2); // recursive case  
}
```

This is correct, but calling `fib(5)` recomputes `fib(2)` three separate times, and `fib(1)/fib(0)` even more often — the same subproblem is solved over and over. This wasted repeated work is called an overlapping subproblem, and it is precisely the situation dynamic programming (Part XV) is built to fix, usually by storing (“memoizing”) each subproblem's answer the first time it's computed.

| n  | Calls made by naive fib(n) | Rough growth    |
|----|----------------------------|-----------------|
| 10 | 177                        | small           |
| 30 | ≈ 2.7 million              | already slow    |
| 40 | ≈ 331 million              | several seconds |
| 50 | ≈ 40 billion               | far too slow    |

#### **Preview: this is fixable without changing the recursion's shape**

Adding a single array to cache each `fib(n)` the first time it's computed (memoization) turns this from exponential into  $O(n)$  — same recursive structure, radically different complexity. Part XV covers this in full; the point here is purely to build the instinct for recognizing overlapping subproblems when you see them.

### 2.2 Sum of an array, recursively

```
int sum(vector<int>& a, int i) {  
    if (i == (int)a.size()) return 0;           // base case: nothing  
    left  
    return a[i] + sum(a, i + 1);               // recursive case  
}  
// call as: sum(a, 0)
```

## 2.3 Reversing a string, recursively

```
string reverseStr(string s) {  
    if (s.size() <= 1) return s;               // base case  
    return reverseStr(s.substr(1)) + s[0];     // recursive case  
}
```

### ⚠ This version is correct but wasteful

`s.substr(1)` copies the whole remaining string on every call, making this  $O(n^2)$  time overall instead of  $O(n)$ . In a contest, prefer passing an index (as in `sum` above) over slicing strings/vectors at every recursive step — indices are  $O(1)$  to pass; substrings are not.

## 2.4 Tree recursion

Fibonacci above is an example of tree recursion: a call that makes more than one recursive call of its own, branching outward like a tree rather than a single chain. Linear recursion (factorial, array sum) makes exactly one recursive call per invocation and traces out a straight line down the stack; tree recursion traces out an actual tree, and its total number of calls can grow exponentially with input size if left unchecked — worth noticing immediately, since it directly predicts whether naive recursion will be fast enough for a given  $n$ .

## Practice — Chapter 2

### Problem 2.1 — Power function [Level 1]

Compute  $x^n$  recursively in  $O(\log n)$  instead of the obvious  $O(n)$  loop.

Hint 1:  $x^n = (x^{(n/2)})^2$  when  $n$  is even, and  $x * x^{(n-1)}$  when  $n$  is odd.

Hint 2: Compute the recursive half only once and square it — don't call the recursive function twice.

Key observation: Halving  $n$  each time, rather than decrementing it by 1, is what turns  $O(n)$  recursion into  $O(\log n)$  recursion — this same halving idea reappears as binary exponentiation in Part XVI.

Approach: `power(x, n)`: if  $n == 0$  return 1; `half = power(x, n/2)`; if  $n$  even return `half*half`; else return `half*half*x`.

Complexity:  $O(\log n)$  time.

## Problem 2.2 — Check if a string is a palindrome [Level 1]

Write a recursive function that checks whether a string reads the same forwards and backwards.

Hint 1: Compare the first and last characters; if they match, recurse on the substring with both ends removed.

Approach: `isPal(s, l, r)`: if  $l \geq r$  return true; if `s[l] != s[r]` return false; return `isPal(s, l+1, r-1)`.

Complexity:  $O(n)$  time when passing indices rather than substrings.

## Chapter 3 — Recursion on Arrays and Strings

### 3.1 A worked dry run: array maximum

```
int findMax(vector<int>& a, int i) {
    if (i == (int)a.size() - 1) return a[i];    // base case: last
    element
    int restMax = findMax(a, i + 1);
    return max(a[i], restMax);
}
```

Array: [3, 7, 2, 9, 4], called as findMax(a, 0).

| Call          | Waits for              | Returns (on the way back up) |
|---------------|------------------------|------------------------------|
| findMax(a, 0) | max(3, restMax)        | max(3, 9) = 9                |
| findMax(a, 1) | max(7, restMax)        | max(7, 9) = 9                |
| findMax(a, 2) | max(2, restMax)        | max(2, 9) = 9                |
| findMax(a, 3) | max(9, restMax)        | max(9, 4) = 9                |
| findMax(a, 4) | base case, i == size-1 | returns a[4] = 4             |

Notice the two distinct phases visible in the trace: an unwinding-downward phase (calls 0 through 4, each waiting on the next) and a returning-upward phase (call 4 resolves first, then 3, then 2, then 1, then 0). Every non-trivial recursive trace has both phases — reading a recursive function's code only shows you the downward phase; dry-running it by hand is what reveals the upward one.

### 3.2 Recursion on strings: counting occurrences

```
int countChar(string& s, int i, char target) {  
    if (i == (int)s.size()) return 0;           // base  
    case  
    int rest = countChar(s, i + 1, target);  
    return rest + (s[i] == target ? 1 : 0);  
}
```

### 3.3 Recursion that returns more than one piece of information

Recursive functions aren't limited to returning a single number — they can return a pair, a vector, or fill in a reference parameter, exactly like any other function (Part I, Chapter 8).

```
// Returns {minimum, maximum} of a[i..end] in one pass  
pair<int,int> minMax(vector<int>& a, int i) {  
    if (i == (int)a.size() - 1) return {a[i], a[i]}; // base cas  
e  
    auto [restMin, restMax] = minMax(a, i + 1);  
    return {min(a[i], restMin), max(a[i], restMax)};  
}
```

#### ✓ How to design a recursive function from scratch

Ask three questions in order: (1) What is the smallest version of this problem I can answer directly — the base case? (2) If I already had the answer for a slightly smaller input, how would I use it to build the answer for the current input — the recursive case? (3) Does every recursive call actually move strictly closer to the base case? If you can answer all three, the function is very likely correct — resist the urge to trace the whole recursion tree in your head while designing it.

## Practice — Chapter 3

### Problem 3.1 — Check if an array is sorted [Level 1]

Write a recursive function that checks whether an array is sorted in non-decreasing order.

Hint 1: Base case: an array of size 0 or 1 is trivially sorted.

Hint 2: Recursive case: the array is sorted if  $a[0] \leq a[1]$  AND the rest (from index 1 onward) is sorted.

Approach: `isSorted(a, i)`: if  $i == a.size()-1$  return true; return  $a[i] \leq a[i+1] \ \&\& \ isSorted(a, i+1)$ .

Complexity:  $O(n)$  time.

### Problem 3.2 — Recursively remove duplicates from a sorted string [Level 2]

Given a sorted string, produce a new string with consecutive duplicate characters removed, using recursion only (no loops).

Hint 1: Compare  $s[0]$  and  $s[1]$ . If they're equal, the first character contributes nothing new — recurse on `s.substr(1)` directly.

Hint 2: If they differ,  $s[0]$  is definitely part of the answer — prepend it to the recursive result on the rest.

Approach: If  $s$  is empty or size 1, return  $s$ . If  $s[0] == s[1]$ , return `dedupe(s.substr(1))`. Else return  $s[0] + \text{dedupe}(s.substr(1))$ .

Complexity:  $O(n)$  calls, though `substr` copying makes the naive version  $O(n^2)$  — passing an index avoids this, same as Section 2.3.

## Chapter 4 — Complexity of Recursion

### 4.1 Time complexity: count the calls

The time complexity of a recursive function is the total amount of work done across every call it makes, not just one call in isolation. Two ingredients determine it: how many recursive calls each invocation makes, and how much the input shrinks per call.

| Shape                          | Example                   | Recursive calls per invocation | Typical complexity     |
|--------------------------------|---------------------------|--------------------------------|------------------------|
| Linear recursion               | factorial, array sum      | 1                              | $O(n)$                 |
| Halving recursion              | binary search, fast power | 1 (but input halves)           | $O(\log n)$            |
| Binary tree recursion          | naive Fibonacci           | 2                              | $O(2^n)$ — exponential |
| Binary recursion, halved input | merge sort                | 2 (each on half the input)     | $O(n \log n)$          |

### 4.2 Recurrence relations, informally

A recurrence relation expresses a function's running time  $T(n)$  in terms of the running time on smaller inputs. Naive Fibonacci gives  $T(n) = T(n-1) + T(n-2) + O(1)$  — each call spawns two calls on slightly smaller inputs, and since the inputs barely shrink ( $n-1$  and  $n-2$ , not  $n/2$ ), the number of calls grows exponentially before hitting a base case. Merge sort (Part VI) gives  $T(n) = 2T(n/2) + O(n)$  — two calls, but each on half the input, which is why it resolves to  $O(n \log n)$  instead of exploding. You do not need to solve recurrences formally (the Master Theorem, taught in some university courses, does this rigorously) to build strong intuition — the pattern to notice is simply: does the input shrink by a constant amount, or by a constant factor?

**💡 Constant amount vs. constant factor — why it matters so much**

Shrinking by a constant amount each call ( $n \rightarrow n-1$ ) means roughly  $n$  calls deep before hitting the base case — that's where linear recursion and unpruned tree recursion both start, but tree recursion additionally branches, multiplying that depth into an exponential call count.

Shrinking by a constant factor ( $n \rightarrow n/2$ ) means only about  $\log_2 n$  calls deep — that's the entire reason binary search and merge sort stay fast even on huge inputs.

### 4.3 Space complexity: the call stack counts too

Recursive space complexity is usually dominated by stack depth — the maximum number of stack frames alive at once — not by the total number of calls made over the function's whole run. Linear recursion on  $n$  elements reaches a maximum depth of  $O(n)$ , even though later frames have already returned and freed their memory by the time deeper ones are still open. Halving recursion reaches a maximum depth of only  $O(\log n)$ .

```
// Stack depth here is O(n): n nested calls are alive simultaneously
// at the deepest point, even though total calls made is also just n.
int sum(vector<int>& a, int i) {
    if (i == (int)a.size()) return 0;
    return a[i] + sum(a, i + 1);
}
```

**⚠️ Recursion depth vs. the number of recursive calls**

These are two different quantities and it's a common confusion point: naive Fibonacci makes an exponential number of calls, but its maximum depth at any instant is only  $O(n)$  (the deepest branch, always calling `fib(n-1)` repeatedly). Time complexity cares about total calls; space complexity (via the call stack) cares about maximum depth.

 **Quick Revision**

- Time complexity of recursion = total work across every call; look at branching factor and how much the input shrinks per call.
- Shrinking input by a constant amount ( $n-1$ ) tends toward  $O(n)$  or, with branching, exponential; shrinking by a constant factor ( $n/2$ ) tends toward  $O(\log n)$  or  $O(n \log n)$ .
- Space complexity of recursion is usually the maximum call-stack depth, not the total number of calls made.
- Naive recursive Fibonacci is the canonical example of exponential blow-up from overlapping subproblems — fixed properly in Part XV (Dynamic Programming).

## Chapter 5 — Backtracking Fundamentals: Subsets and Subsequences

### 5.1 What backtracking adds to recursion

In one sentence: backtracking is recursion used to explore every possible choice at each step, undoing a choice after exploring it so the next choice can be tried cleanly. The discipline is always the same three moves: choose (make a decision), explore (recurse with that decision in effect), un-choose (undo the decision before trying the next option). That third step — undoing — is the only thing that distinguishes backtracking from recursion in general, and it's easy to forget because it happens after the recursive call returns.

### 5.2 Generating all subsets

For each element of the input, there are exactly two choices: include it in the current subset, or don't. Making that choice explicitly, for every element in turn, generates every one of the  $2^n$  possible subsets.

```
vector<int> current;
vector<vector<int>> allSubsets;

void subsets(vector<int>& a, int i) {
    if (i == (int)a.size()) {
        allSubsets.push_back(current);    // record a completed sub
set
        return;
    }
    // Choice 1: exclude a[i]
    subsets(a, i + 1);

    // Choice 2: include a[i]
    current.push_back(a[i]);              // choose
    subsets(a, i + 1);                    // explore
    current.pop_back();                   // un-choose (backtrack)
}
// call as: subsets(a, 0)
```

### 5.3 Dry run on [1, 2]

| Call                          | current before | Action                           | current after |
|-------------------------------|----------------|----------------------------------|---------------|
| subsets([1,2], 0)             | []             | exclude 1 → subsets(i=1)         | []            |
| subsets([1,2], 1)             | []             | exclude 2 → subsets(i=2)         | []            |
| subsets([1,2], 2)             | []             | base case: record []             | []            |
| (back in i=1)                 | []             | include 2 → push 2, subsets(i=2) | [2]           |
| subsets([1,2], 2)             | [2]            | base case: record [2]            | [2]           |
| (back in i=1)                 | [2]            | pop 2 (backtrack)                | []            |
| (back in i=0)                 | []             | include 1 → push 1, subsets(i=1) | [1]           |
| subsets([1,2], 1)             | [1]            | exclude 2 → subsets(i=2)         | [1]           |
| subsets([1,2], 2)             | [1]            | base case: record [1]            | [1]           |
| (back in i=1)                 | [1]            | include 2 → push 2, subsets(i=2) | [1,2]         |
| subsets([1,2], 2)             | [1,2]          | base case: record [1,2]          | [1,2]         |
| (unwinds, popping back to []) | —              | —                                | []            |

Final `allSubsets`: [], [2], [1], [1,2] — all four subsets of a two-element set, each produced exactly once. Every `push_back` is undone by a matching `pop_back` once that branch is fully explored — current is always back to exactly what it was before entering a call, by the time that call returns.

**⚠ The single most common backtracking bug**

Forgetting the “un-choose” step (`pop_back()` here) leaves stale choices in current that contaminate every subsequent branch. If your backtracking output looks like it's accumulating extra, wrong elements as it goes, check first whether every push/insert has a matching pop/erase on every code path — including any early return for pruning (Chapter 7).

## Practice — Chapter 5

### Problem 5.1 — Print all subsets as strings [Level 1]

Given a string, print every subsequence (including the empty one).

Hint 1: Identical structure to the subset code above — a string index plays the role of the array index.

Approach: Same include/exclude choice per character, building up a string instead of a vector.

Complexity:  $O(2^n)$  subsequences,  $O(n)$  work to build each —  $O(n \cdot 2^n)$  total.

### Problem 5.2 — Subsets that sum to a target [Level 3]

Given an array and a target sum, count how many subsets sum exactly to the target.

Hint 1: Same include/exclude recursion, but track the running sum instead of building the actual subset.

Hint 2: Add a prune: if the running sum already exceeds the target (and all remaining elements are positive), stop exploring that branch early.

Key observation: This is the subset-sum problem, which has overlapping subproblems for repeated (index, remaining target) pairs — it becomes a DP problem (Part XV) once memoized.

Complexity:  $O(2^n)$  without memoization;  $O(n \times \text{target})$  once it's rewritten as DP.

## Chapter 6 — Permutations and Combinations

### 6.1 Permutations: order matters

A permutation is an arrangement of all  $n$  elements where order matters — there are  $n!$  of them. The backtracking approach: at each position, try every element that hasn't been used yet, mark it used, recurse for the next position, then unmark it.

```
vector<int> current;
vector<bool> used;
vector<vector<int>> allPerms;

void permute(vector<int>& a) {
    if ((int)current.size() == (int)a.size()) {
        allPerms.push_back(current);
        return;
    }
    for (int i = 0; i < (int)a.size(); i++) {
        if (used[i]) continue;           // skip elements already
y        placed
        used[i] = true;                  // choose
        current.push_back(a[i]);
        permute(a);                      // explore
        current.pop_back();              // un-choose
        used[i] = false;
    }
}
// call as: used.assign(a.size(), false); permute(a);
```

#### **std::next\_permutation exists — know both**

In a contest, generating permutations in lexicographic order via repeated calls to `next_permutation` (Part IV) is usually faster to write than backtracking by hand. Learn the backtracking version anyway: it's the general pattern that extends to permutations with constraints (skip invalid placements, as N-Queens does in Chapter 7), which `next_permutation` alone cannot express.

## 6.2 Combinations: order doesn't matter

A combination of size  $k$  picks  $k$  elements from  $n$  where order is irrelevant — choosing  $\{1,2\}$  is the same combination as  $\{2,1\}$ . The standard trick to avoid generating the same combination twice: only ever consider elements at or after the index you started from, so each combination is built in one fixed relative order.

```
vector<int> current;
vector<vector<int>> allCombos;

void combine(vector<int>& a, int start, int k) {
    if ((int)current.size() == k) {
        allCombos.push_back(current);
        return;
    }
    for (int i = start; i < (int)a.size(); i++) {
        current.push_back(a[i]);           // choose
        combine(a, i + 1, k);              // explore – next start
        current.pop_back();                // never revisits earlier
                                           // indices
    }
}
// call as: combine(a, 0, k)
```

## 6.3 Why combinations don't need a "used" array

Permutations need a `used[]` array because any unused element can go in any position — the set of valid next choices depends on history in a way that isn't captured by a single index. Combinations sidestep this entirely: passing `start = i + 1` already guarantees no index is revisited and no combination is generated more than once (in a different order), so no extra bookkeeping is needed.

|                       | Permutations  | Combinations                      |
|-----------------------|---------------|-----------------------------------|
| Order matters?        | Yes           | No                                |
| Count (choose all n)  | $n!$          | — (use k of n instead)            |
| Count (choose k of n) | $n! / (n-k)!$ | $n! / (k! (n-k)!)$ — "n choose k" |
| Needs a used[] array? | Yes           | No — start index suffices         |

## Practice — Chapter 6

### Problem 6.1 — Permutations of a string with duplicate characters [Level 3]

Given a string that may contain repeated characters, print all distinct permutations (no duplicates in the output).

Hint 1: Sort the string first. At each position, skip an element if it equals the previous element at the same recursion level AND the previous one hasn't been used yet in this branch.

Key observation: Without the skip rule, swapping two equal characters produces two identical permutations that look different to the algorithm but are the same string — sorting plus the adjacency check prevents generating a duplicate in the first place, rather than filtering duplicates afterward.

Complexity: Up to  $O(n \times n!)$  in the worst case (all distinct characters); far fewer with repeats.

### Problem 6.2 — Combination sum [Level 4]

Given an array of distinct positive integers and a target, find all combinations (elements may be reused any number of times) that sum to the target.

Hint 1: Same start-index technique as Section 6.2, but pass  $i$  (not  $i+1$ ) into the recursive call when an element can be reused.

Hint 2: Prune: if the running sum exceeds the target, stop exploring that branch.

Approach: `combSum(a, start, target, current)`: if `target==0`, record `current`. For `i` from `start` to `n-1`: if `a[i] > target`, skip/break (if sorted); else choose `a[i]`, recurse with `combSum(a, i, target-a[i], ...)` (`i`, not `i+1`, since reuse is allowed), then un-choose.

Complexity: Exponential in the worst case, bounded heavily by the pruning and by target size.

## Chapter 7 — N-Queens and the Art of Pruning

### 7.1 The problem

Place  $n$  queens on an  $n \times n$  chessboard so that no two queens attack each other — no two share a row, a column, or a diagonal. This is the textbook example of backtracking with pruning, because the constraints (no shared row/column/diagonal) let you rule out huge swaths of the search space long before reaching a complete, invalid placement.

### 7.2 The key simplification: one queen per row

Since no two queens may share a row, there is exactly one queen in each row. That collapses the problem from “which  $n^2$  cells contain queens” down to “which column does the queen in row  $r$  go in, for each row  $r$ ” — a much smaller decision per row, and exactly the shape backtracking handles well: one choice per level of recursion.

```

int n;
vector<int> col; // col[r] = column of the queen in row r
int solutions = 0;

bool isSafe(int row, int c) {
    for (int r = 0; r < row; r++) {
        int prevCol = col[r];
        if (prevCol == c) return false; // same column
        if (abs(prevCol - c) == abs(r - row)) return false; // same diagonal
    }
    return true;
}

void solve(int row) {
    if (row == n) { solutions++; return; } // placed a queen in every row
    for (int c = 0; c < n; c++) {
        if (!isSafe(row, c)) continue; // PRUNE: skip invalid choices
        col[row] = c; // choose
        solve(row + 1); // explore
        // no explicit un-choose needed here – col[row] is simply
        // overwritten on the next iteration or the next call
    }
}
// call as: col.assign(n, -1); solve(0);

```

### 7.3 Why isSafe is the pruning step

Without `isSafe`, this code would place queens in all  $n^n$  row/column combinations and check validity only at the end — enormously wasteful, since an invalid placement in row 2 makes every completion of rows 3..n-1 pointless to even generate. `isSafe` is called before recursing deeper, so an unsafe column is rejected immediately — the recursive call for that branch is never made at all. This is the essence of pruning:

cut a branch as early as possible, not after fully exploring it.

✓ **Recognizing where to prune in general**

Ask: “Can I tell, from a partial (incomplete) choice, that no completion of it can ever be valid?” If yes, check that condition before recursing, not after. In N-Queens: a queen placed unsafely in row 2 can never be fixed by anything chosen in later rows, so it's checked immediately. The same idea drove the running-sum prune in Problem 5.2 and the  $\text{sum} > \text{target}$  prune in Problem 6.2.

## 7.4 How much pruning actually saves

| n  | Unpruned search space ( $n^n$ ) | Actual solutions found |
|----|---------------------------------|------------------------|
| 4  | 256                             | 2                      |
| 8  | 16,777,216                      | 92                     |
| 10 | 10,000,000,000                  | 724                    |

Pruning doesn't just skip a few bad branches at the very end — it collapses entire subtrees the moment a conflict is detected, which is why the algorithm above finishes almost instantly for these sizes despite the raw search space being astronomically larger than the solution count.

**📌 Quick Revision**

- Backtracking = choose, explore (recurse), un-choose — the un-choose step is the one beginners most often forget.
- Subsets/subsequences: an include/exclude choice per element,  $2^n$  outcomes.
- Permutations need a used[] array (any unused element can go anywhere); combinations use a start index instead (no revisiting earlier indices), which naturally avoids duplicate orderings.
- Prune by checking a partial choice's validity BEFORE recursing deeper, not after completing it — this is what makes backtracking fast in practice despite an exponential worst case.
- N-Queens' one-queen-per-row simplification is a general technique: look for a constraint that fixes one dimension of the choice, reducing what backtracking has to decide per level.

## Practice — Chapter 7

### Problem 7.1 — Sudoku validity check as a warm-up [Level 2]

Before attempting a Sudoku solver, write `isSafe(board, row, col, digit)` checking the row, column, and 3x3 box constraints.

Hint 1: This is structurally the same kind of check as N-Queens' `isSafe` — a local validity test used to prune before placing a digit.

Approach: Check the digit doesn't already appear in `board[row][*]`, `board[*][col]`, or the 3x3 box containing (row, col).

Complexity:  $O(1)$  per check (fixed 9x9 board).

### Problem 7.2 (harder) — N-Queens: print one valid board instead of counting all [Level 4]

Modify the N-Queens backtracking to stop and print the first valid board found, instead of counting every solution.

Hint 1: Have `solve()` return a bool. Return true immediately once a full valid placement (`row == n`) is found, and propagate that true up through the loop to stop trying further columns.

**Key observation:** Counting all solutions requires exploring the entire search tree; stopping at the first solution lets pruning cut the work dramatically further, since most branches are abandoned the instant a valid answer is located anywhere beneath them.

**Complexity:** Still exponential worst case, but typically far faster in practice than counting all solutions.

## Part X — Master Revision

### Must Remember

#### Quick Revision

- Every recursive function needs a base case that stops it and a recursive case that shrinks the input toward that base case.
- The call stack holds one frame per active call; calls resolve in reverse order of when they were made (last called, first to finish).
- Shrinking the input by a constant amount tends toward  $O(n)$  (or exponential, with branching); shrinking by a constant factor tends toward  $O(\log n)$  or  $O(n \log n)$ .
- Recursive space complexity is usually the maximum call-stack depth, not the total number of calls made.
- Backtracking is recursion plus a strict choose  $\rightarrow$  explore  $\rightarrow$  un-choose discipline; forgetting to un-choose is the most common bug.
- Prune by rejecting an invalid partial choice BEFORE recursing on it — that's what keeps exponential-looking algorithms fast in practice.

### Common Mistakes Recap

| Mistake                                                                                                      | Fix                                                                                               |
|--------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| Missing or unreachable base case                                                                             | Trace the smallest possible input by hand and confirm it stops without recursing                  |
| Passing substrings/subvectors by value at every call                                                         | Pass an index/range into the original container instead — avoids $O(n)$ copying per call          |
| Forgetting to undo a choice ( <code>pop_back</code> , <code>unmark used</code> ) after a backtracking branch | Make sure every choose has a matching un-choose on every exit path, including early returns       |
| Checking validity only after a placement is complete                                                         | Move the validity check before the recursive call — that's the prune                              |
| Confusing total call count with maximum stack depth                                                          | Time complexity $\approx$ total calls; space complexity $\approx$ deepest simultaneous call chain |

| Mistake                                                                 | Fix                                                                                      |
|-------------------------------------------------------------------------|------------------------------------------------------------------------------------------|
| Recurring deeper than the stack can hold (large $n$ , linear recursion) | Consider an iterative rewrite, or increase recursion limits only if the judge permits it |

## Complexity Table

| Pattern                                                   | Time                         | Space (stack depth) |
|-----------------------------------------------------------|------------------------------|---------------------|
| Linear recursion (factorial, array sum)                   | $O(n)$                       | $O(n)$              |
| Halving recursion (binary search, fast power)             | $O(\log n)$                  | $O(\log n)$         |
| Naive tree recursion (Fibonacci)                          | $O(2^n)$                     | $O(n)$              |
| Divide & conquer, halved input, linear merge (merge sort) | $O(n \log n)$                | $O(\log n)$         |
| Subsets / include-exclude backtracking                    | $O(2^n)$                     | $O(n)$              |
| Permutations backtracking                                 | $O(n \times n!)$             | $O(n)$              |
| N-Queens backtracking (with pruning)                      | well below $n^n$ in practice | $O(n)$              |

## Interview-Style Questions

- Why does every correct recursive function need a base case, and what happens at runtime if it's missing?
- Explain the difference between the total number of calls a recursive function makes and its maximum recursion depth, with an example where they differ significantly.
- Why is naive recursive Fibonacci exponential, and what single change fixes it without altering the recursive structure?

- In backtracking, why must a choice be undone after exploring it, even though the recursive call has already returned?
- Why does pruning before a recursive call matter more than checking validity after the call returns?

## Cheat Sheet — Part X

```
// Generic recursion skeleton
ReturnType f(Params) {
    if (/* base case */) return baseAnswer;
    // ... recursive case: call f on a smaller input, combine results
}

// Subsets (include/exclude)
void subsets(vector<int>& a, int i) {
    if (i == (int)a.size()) { record(current); return; }
    subsets(a, i + 1);           // exclude
    current.push_back(a[i]);
    subsets(a, i + 1);           // include
    current.pop_back();
}

// Permutations (used[] array)
void permute(vector<int>& a) {
    if ((int)current.size() == (int)a.size()) { record(current); return; }
    for (int i = 0; i < (int)a.size(); i++) {
        if (used[i]) continue;
        used[i] = true; current.push_back(a[i]);
        permute(a);
        current.pop_back(); used[i] = false;
    }
}
```

```
// Combinations (start index, no reuse)
void combine(vector<int>& a, int start, int k) {
    if ((int)current.size() == k) { record(current); return; }
    for (int i = start; i < (int)a.size(); i++) {
        current.push_back(a[i]);
        combine(a, i + 1, k);
        current.pop_back();
    }
}

// N-Queens skeleton
void solve(int row) {
    if (row == n) { solutions++; return; }
    for (int c = 0; c < n; c++) {
        if (!isSafe(row, c)) continue;    // prune
        col[row] = c;
        solve(row + 1);
    }
}
```

### Coming up in Part XI

Part XI — Trees and BST moves from linear/branching recursion into recursion on a genuine tree-shaped structure: traversals (preorder, inorder, postorder, level order), height and diameter, and binary search tree operations. Almost every tree algorithm is a direct application of the recursive design process from Chapter 3 of this Part — base case at a null node, recursive case combining the left and right subtrees— so the habits built here transfer directly.